

Processamento de Dados - Investimentos Públicos por Programa e Região (Sefaz-CE)

Paulo Icaro

Objetivo e Estrutura dos Dados

A rotina desenvolvida visa realizar a devida padronização nos dados de investimentos por programa e/ou região que municiam os modelos trabalhados no projeto.

A imagem a seguir representa a estrutura dos dados disponibilizados em cada uma das planilhas baixadas na página do [SIOF](#). Cada planilha representa um **Tipo** de investimento: Equipamentos, Obras e Total. Além disso a disponibilidade efetiva das informações se dá a partir do ano de 2016 em razão das informações no período de 2013 a 2015 não estarem disponíveis por região de planejamento. Tratado de informações efetivas, cada planilha contém oito campos. Destes, as colunas interesse são:

- Código
- Descrição
- Empenhado
- Pago

No campo Código, os investimentos estão classificados em Programa e Região. Assim cabe ao pesquisador definir como ele deseja o resultado final: apenas Programa, apenas Região ou ambos.

Levando em conta os pontos mencionados e que os dados são cumulativos, o objetivo dessa rotina é coletar as informações referentes Programa e/ou Região considerando somente os valores de investimentos Empenhado e Pago. Os tópicos a seguir detalham cada etapa da rotina.



ESTADO DO CEARÁ EXECUÇÃO ORÇAMENTÁRIA - 2018

LEI Nº 16.468, de 19/12/2017
Consolidado por Programa e Região

Acumulado até: ABRIL
PERCENTUAL S/ AUTORIZADO

Código	Descrição	Lei	Lei + Cred.	Empenhado	Pago	% Emp.	% Pago
001	GESTÃO DE RISCOS E DESASTRES	4.010.000,00	15.347.768,23	1.852.412,58	1.852.412,58	7,62	5,53
03	GRANDE FORTALEZA	0,00	1.852.412,58	1.852.412,58	1.852.412,58	100,00	100,00
15	ESTADO DO CEARÁ	4.010.000,00	13.495.355,65	0,00	0,00	0,00	0,00
003	SEGURANÇA PÚBLICA INTEGRADA	8.377.488,00	10.333.641,97	3.455.314,84	2.441.371,99	7,62	5,53
01	CARIRI	7.000,00	124.007,36	118.007,36	118.007,36	95,16	95,16
02	CENTRO SUL	2.000,00	1.000,00	0,00	0,00	0,00	0,00
03	GRANDE FORTALEZA	2.176.500,00	3.576.976,77	2.495.242,99	1.719.161,04	69,76	48,06
04	LITORAL LESTE	13.000,00	8.000,00	0,00	0,00	0,00	0,00
05	LITORAL NORTE	13.000,00	4.000,00	0,00	0,00	0,00	0,00
06	LITORAL OESTE / VALE DO CURU	2.829.753,00	2.490.874,05	442.933,31	442.933,31	17,78	17,78
07	MACIÇO DO BATURITÉ	1.000,00	0,00	0,00	0,00	0,00	0,00
08	SERRA DA IBIAPABA	23.557,00	22.557,00	0,00	0,00	0,00	0,00
09	SERTÃO CENTRAL	19.000,00	393.000,00	0,00	0,00	0,00	0,00
10	SERTÃO DE CANINDÉ	4.000,00	56.915,48	53.915,48	53.915,48	94,73	94,73
11	SERTÃO DE SOBRAL	22.000,00	13.000,00	0,00	0,00	0,00	0,00
12	SERTÃO DOS CRATEÚS	2.824.753,00	2.481.374,06	255.215,55	17.354,65	10,29	0,70
13	SERTÃO DOS INHAMUNS	43.368,00	42.368,00	0,00	0,00	0,00	0,00
14	VALE DO JAGUARIBE	17.000,00	687.257,89	90.000,15	90.000,15	13,10	13,10
15	ESTADO DO CEARÁ	381.557,00	432.311,36	0,00	0,00	0,00	0,00
004	INFRAESTRUTURA E GESTÃO DO SISTEMA PENITENCIÁRIO	32.464.265,00	34.254.306,03	207.431,46	207.431,46	7,62	5,53
01	CARIRI	5.000,00	0,00	0,00	0,00	0,00	0,00
02	CENTRO SUL	10.000,00	0,00	0,00	0,00	0,00	0,00
03	GRANDE FORTALEZA	7.494.058,00	22.322.890,64	207.431,46	207.431,46	0,93	0,93
04	LITORAL LESTE	2.100,00	0,00	0,00	0,00	0,00	0,00
05	LITORAL NORTE	10.000,00	0,00	0,00	0,00	0,00	0,00
06	LITORAL OESTE / VALE DO CURU	5.000,00	0,00	0,00	0,00	0,00	0,00
07	MACIÇO DO BATURITÉ	500,00	0,00	0,00	0,00	0,00	0,00
08	SERRA DA IBIAPABA	2.402.500,00	2.402.000,00	0,00	0,00	0,00	0,00
09	SERTÃO CENTRAL	100,00	0,00	0,00	0,00	0,00	0,00
10	SERTÃO DE CANINDÉ	100,00	0,00	0,00	0,00	0,00	0,00

Bibliotecas e Arquivos na Pasta

Para execução dessa rotina, duas bibliotecas foram utilizadas:

- **pandas**: biblioteca para manipulação, limpeza e análise de dados.
- **os**: biblioteca padrão do python que permite interagir com o sistema operacional.

Importadas as devidas bibliotecas, os arquivos que serão trabalhados são devidamente mapeados por meio da função **listdir** da biblioteca **os**. Nessa etapa o usuário será questionado sobre como deseja a estrutura final dos dados: Programa, Região ou Programa/Região¹.

```
# ===== #
# === Bibliotecas === #
# ===== #
import pandas as pd
```

¹É válido mencionar que o usuário deve preencher corretamente a informação indicado qual tipo de análise deseja realizar. Caso contrário, a rotina acarretará em um erro.

```
import os
```

```
# ===== #
# === Definindo o tipo de dado desejado === #
# ===== #
info_desired = ''

while info_desired not in {'p', 'r', 'pr', 'rp'}:
    info_desired = input('Como você deseja a base de dados ? Use (P) para Programa,
    ↪ (R) para Região e (PR) para Programa e Região: ').strip().lower()

    if info_desired not in {'p', 'r', 'pr', 'rp'}:
        print('Opção inválida !')
    elif info_desired == 'p':
        print('Tratando as informações por Programa ...')
        break
    elif info_desired == 'r':
        print('Tratando as informações por Região ...')
        break
    else:
        print('Tratando as informações por Programa e Região ...')
        break
```

Um *dataframe*, **dataset_full**, será gerado e irá armazenar todo o conjunto de dados final.

```
# ===== #
# === Arquivos de Dados === #
# ===== #
folder_files = os.listdir('Dataset/Investimentos_Programa_Regiao/')
dataset_full = pd.DataFrame()

# --- Prints --- #
print(*folder_files[0:10], sep = '\n')
```

```
MAI_2024_EQUIP.xlsx
OUT_2022_EQUIP.xlsx
FEV_2015_OBRAS.XLS
JAN_2025_EQUIP.XLS
AGO_2021_TOTAL.xlsx
MAR_2018_EQUIP.xlsx
DEZ_2015_EQUIP.XLS
MAI_2017_EQUIP.xlsx
FEV_2021_TOTAL.xlsx
JUL_2022_TOTAL.xlsx
```

Processamento de Dados

Nesta etapa, uma estrutura *loop* é utilizada realizar a manipulação da rotina, onde cada etapa do tratamento é executado em cada conjunto de informações contidos nas planilhas.

Na leitura dos arquivos, somente as colunas de interesse são coletadas. Além disso, toda e qualquer linha nula é removida desse conjunto. Esse conjunto de dados remodelado recebe o nome de **dataset**.

Ao **dataset** são acrescidos quatro novos campos representando, respectivamente, i) período, ii) tipo de investimento, iii) ano, e iv) mês. Tais informações são extraídas justamente do nome de cada arquivo processado². Esse procedimento se replica nos três casos possíveis, porém a uma pequena modificação é feita no cenário programa/região: os campos **cod_programa** e **programa** também são incluídos.

Após esses tratamentos iniciais, algumas linhas de comando são responsáveis por duas partes cruciais do procedimento. Para cada cenário um procedimento é adotado:

- **Investimentos por Programa:** As linhas que remetem a região de planejamento possuem dois caracteres. Assim, todas estas são identificadas e depois removidas do **dataset**, restando desse modo somente as informações referentes aos programas;
- **Investimentos por Região:** Similar ao caso anterior, agora todas as linhas que representam programa é que são identificadas e removidas;
- **Programa/Região:** Aqui o ajuste é um pouco mais refinado. Inicialmente as linhas que representam programa são identificadas. Os campos **cod_programa** e **programa** são alimentados, respectivamente com as informações dos campos **Código** e **Descrição** apenas nas linhas que foram identificadas. Em seguida, utilizando a função **ffill**, as colunas **cod_programa** e **programa** de todas as linhas que por eliminação representam região, passam a receber o valor imediatamente anterior. Por fim, as linhas que representam programa são eliminadas. A imagem a seguir ajuda a entender o procedimento adotado:

²A padronização dos nomes dos arquivos é ponto crucial na distinção das informações. Por exemplo, para os investimentos em Equipamentos no período de Março de 2020, teria-se **MAR_2020_EQUIP.xlsx**.

1

Código	Descrição	Empenhado	Pago
122	Proteção Social Especial	R\$ 371.379,16	R\$ 371.379,16
01	Cariri	R\$ 371.379,16	R\$ 371.379,16
14	Vale do Jaguaribe	R\$ 0,00	R\$ 0,00
15	Estado do Ceará	R\$ 0,00	R\$ 0,00

2

Código	Descrição	Cod_Programa	Programa	Empenhado	Pago
122	Proteção Social Especial			R\$ 371.379,16	R\$ 371.379,16
01	Cariri			R\$ 371.379,16	R\$ 371.379,16
14	Vale do Jaguaribe			R\$ 0,00	R\$ 0,00
15	Estado do Ceará			R\$ 0,00	R\$ 0,00

3

Código	Descrição	Cod_Programa	Programa	Empenhado	Pago
122	Proteção Social Especial	122	Proteção Social Especial	R\$ 371.379,16	R\$ 371.379,16
01	Cariri			R\$ 371.379,16	R\$ 371.379,16
14	Vale do Jaguaribe			R\$ 0,00	R\$ 0,00
15	Estado do Ceará			R\$ 0,00	R\$ 0,00

4

Código	Descrição	Cod_Programa	Programa	Empenhado	Pago
122	Proteção Social Especial	122	Proteção Social Especial	R\$ 371.379,16	R\$ 371.379,16
01	Cariri	122	Proteção Social Especial	R\$ 371.379,16	R\$ 371.379,16
14	Vale do Jaguaribe	122	Proteção Social Especial	R\$ 0,00	R\$ 0,00
15	Estado do Ceará	122	Proteção Social Especial	R\$ 0,00	R\$ 0,00

5

Cod_Regiao	Regiao	Cod_Programa	Programa	Empenhado	Pago
01	Cariri	122	Proteção Social Especial	R\$ 371.379,16	R\$ 371.379,16
14	Vale do Jaguaribe	122	Proteção Social Especial	R\$ 0,00	R\$ 0,00
15	Estado do Ceará	122	Proteção Social Especial	R\$ 0,00	R\$ 0,00

Cada planilha que passa por esse procedimento representa um dataset que contribui para um dataframe final chamado `dataset_full` contendo todas as informações por ano, mês e tipo de investimento e função.

```
# ===== #
# === Processamento de Dados === #
# ===== #
for x in range(len(folder_files)):

    dataset = pd.read_excel(io = 'Dataset/Investimentos_Programa_Regiao/' +
↪ folder_files[x],
                           header = 10,
                           usecols= 'C, F, K, N',
                           names = ['codigo', 'descricao', 'empenhado', 'pago'],
                           dtype = {'codigo':str})
    dataset = dataset.dropna()
```

```

# ----- #
# --- Programa --- #
# ----- #
if info_desired == 'p':
    dataset = dataset.assign(perodo = folder_files[x][0:8],
                             tipo = folder_files[x][9:14],
                             ano = folder_files[x][4:8],
                             mes = folder_files[x][0:3])

    # --- Substituições --- #
    replacements = {'JAN':'01', 'FEV':'02', 'MAR':'03', 'ABR':'04', 'MAI':'05',
↪ 'JUN':'06', 'JUL':'07', 'AGO':'08', 'SET':'09', 'OUT':'10', 'NOV':'11',
↪ 'DEZ':'12'}
    for old, new in replacements.items():
        dataset['mes'] = dataset['mes'].replace(old,new)

    # --- Identificando linhas de Região --- #
    program_flag = dataset['codigo'].str.len() == 2

    # --- Removendo casos onde a coluna codigo possui 2 caracteres --- #
    dataset = dataset[~program_flag]

    # --- Reordenando e renomeando --- #
    dataset = dataset.reindex(columns = ['perodo', 'ano', 'mes', 'tipo',
↪ 'codigo', 'descricao', 'empenhado', 'pago'])
    dataset.rename(columns = {'descricao':'programa', 'codigo':'cod_programa'},
↪ inplace = True)

# ----- #
# --- Região --- #
# ----- #
if info_desired == 'r':
    dataset = dataset.assign(perodo = folder_files[x][0:8],
                             tipo = folder_files[x][9:14],
                             ano = folder_files[x][4:8],
                             mes = folder_files[x][0:3])

    # --- Substituições --- #
    replacements = {'JAN':'01', 'FEV':'02', 'MAR':'03', 'ABR':'04', 'MAI':'05',
↪ 'JUN':'06', 'JUL':'07', 'AGO':'08', 'SET':'09', 'OUT':'10', 'NOV':'11',
↪ 'DEZ':'12'}
    for old, new in replacements.items():
        dataset['mes'] = dataset['mes'].replace(old,new)

```

```

# --- Identificando linhas de Programa --- #
region_flag = dataset['codigo'].str.len() == 3

# --- Removendo casos onde a coluna código possui 3 caracteres --- #
dataset = dataset[~region_flag]

# --- Reordenando e renomeando --- #
dataset = dataset.reindex(columns = ['periodo', 'ano', 'mes', 'tipo',
↪ 'codigo', 'descricao', 'empenhado', 'pago'])
dataset.rename(columns = {'descricao':'regiao', 'codigo':'cod_regiao'},
↪ inplace = True)

# ----- #
# --- Programa and Região --- #
# ----- #
if info_desired == 'rp' or info_desired == 'pr':
    dataset = dataset.assign(cod_programa = None,
↪                             programa = None,
↪ # Add empty column programa
                             periodo = folder_files[x][0:8],
                             tipo = folder_files[x][9:14],
                             ano = folder_files[x][4:8],
                             mes = folder_files[x][0:3])

# --- Substituições --- #
replacements = {'JAN':'01', 'FEV':'02', 'MAR':'03', 'ABR':'04', 'MAI':'05',
↪ 'JUN':'06', 'JUL':'07', 'AGO':'08', 'SET':'09', 'OUT':'10', 'NOV':'11',
↪ 'DEZ':'12'}
for old, new in replacements.items():
    dataset['mes'] = dataset['mes'].replace(old,new)

# --- Identificando linhas de Programa --- #
program_flag = dataset['codigo'].str.len() == 3

# --- Preenchendo colunas --- #
dataset.loc[program_flag, 'cod_programa'] = dataset['codigo']
dataset.loc[program_flag, 'programa'] = dataset['descricao']
dataset[['cod_programa', 'programa']] =
↪ dataset[['cod_programa', 'programa']].ffill()

# --- Removendo casos onde a coluna código possui 3 caracteres --- #
dataset = dataset[~program_flag]

# --- Reordenando e renomeando --- #
dataset = dataset.reindex(columns = ['periodo', 'ano', 'mes', 'tipo',
↪ 'codigo', 'descricao', 'cod_programa', 'programa', 'empenhado', 'pago'])

```

```

        dataset.rename(columns = {'descricao':'regiao', 'codigo':'cod_regiao'},
↪ inplace = True)

# ----- #
# --- Empilhando os dados --- #
# ----- #
if x == 0:
    dataset_full = dataset
else:
    dataset_full = pd.concat([dataset_full, dataset])

# --- Prints --- #
print(dataset_full.loc[:, ~dataset_full.columns.isin(['regiao',
↪ 'programa'])].head(10))

```

	periodo	ano	mes	tipo	cod_regiao	cod_programa	empenhado	pago
2	MAI_2024	2024	05	EQUIP	03	101	0.0	0.0
3	MAI_2024	2024	05	EQUIP	15	101	0.0	0.0
6	MAI_2024	2024	05	EQUIP	15	102	0.0	0.0
8	MAI_2024	2024	05	EQUIP	01	112	0.0	0.0
9	MAI_2024	2024	05	EQUIP	02	112	0.0	0.0
10	MAI_2024	2024	05	EQUIP	03	112	0.0	0.0
11	MAI_2024	2024	05	EQUIP	04	112	0.0	0.0
12	MAI_2024	2024	05	EQUIP	05	112	0.0	0.0
13	MAI_2024	2024	05	EQUIP	06	112	0.0	0.0
14	MAI_2024	2024	05	EQUIP	07	112	0.0	0.0

Ajustes para Dados Cumulativos

Nessa seção da rotina, o objetivo principal é chegar ao investimento efetivo que se deu em determinado período. Dessa forma, a ideia é sempre tomar a diferença entre os períodos adotando a seguinte regra:

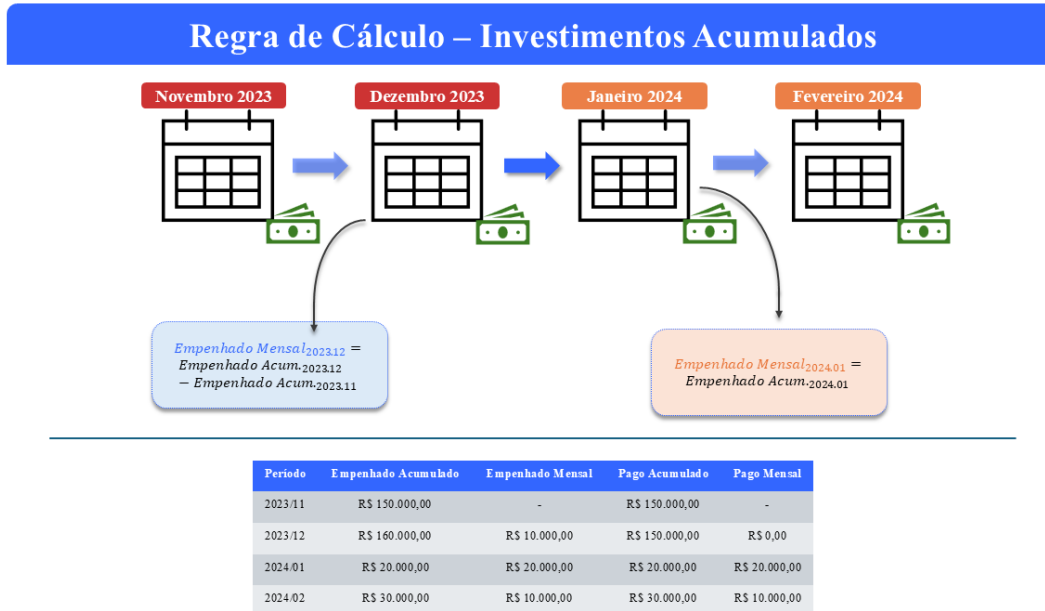
$$invest_t = invest_acum_t - invest_acum_{t-1} \quad (t \neq 1)$$

É interessante ressaltar que para chegar nesse cálculo o dataframe **dataset_full** precisa estar devidamente ordenado, evitando assim variações entre períodos, tipos de investimento programas e regiões diferentes. Em vista dessa necessidade, a função **sort_values** desempenha esse papel de ordenamento a depender do cenário selecionado pelo pesquisador:

- **Investimentos por Programa:** Código do Programa, Tipo, Ano e Mês;
- **Investimentos por Região:** Código da Região, Tipo, Ano e Mês;
- **Programa/Região:** Tipo, Código da Região, Código do Programa, Ano e Mês;

No **dataset_full** duas novas colunas representando o investimento mensal, **empenhado_mensal** e **pago_mensal**, são adicionadas, ao passo que os campos **empenhado** e **pago** passam a se chamar **empenhado_acumulado** e **pago_acumulado**.

Para valores que representam o mês de janeiro, uma regra em looping foi gerada. Uma vez que os dados estão devidamente ordenados e não há possibilidade alguma de conflito de informações, ao comparar a linha atual com a anterior, se a diferença entre os meses for superior a 1, então tem-se a confirmação de um cenário onde a linha atual representa janeiro e a linha anterior representa dezembro do ano anterior. Nesse caso, a rotina entende que o valor mensal é o mesmo valor acumulado. Nos demais cenários a regra padrão é adotada. O infográfico a seguir ajuda a compreender a regra adotada.



```
# ===== #
# === Ajustes para Dados Cumulativos === #
# ===== #

# --- Ordenamento --- #
if info_desired == 'p':
    dataset_full = dataset_full.sort_values(by = ['cod_programa', 'tipo', 'ano',
↪ 'mes']).reset_index(drop = True)
```

```

elif info_desired == 'r':
    dataset_full = dataset_full.sort_values(by = ['cod_regiao', 'tipo', 'ano',
↳ 'mes']).reset_index(drop = True)
else:
    dataset_full = dataset_full.sort_values(by = ['tipo', 'cod_regiao',
↳ 'cod_programa', 'ano', 'mes']).reset_index(drop = True)

# --- Ajuste nos dados cumulativos --- #
dataset_full['empenhado_mensal'] = dataset_full['empenhado'] -
↳ dataset_full['empenhado'].shift(1) # Inserting adjusted values
dataset_full['pago_mensal'] = dataset_full['pago'] - dataset_full['pago'].shift(1)
↳ # Inserting adjusted values

# --- Looping de ajuste para valores com datas truncadas --- #
for i in range(len(dataset_full)):
    if i == 0:
        dataset_full.loc[0, 'empenhado_mensal'] = dataset_full.loc[0, 'empenhado']
        dataset_full.loc[0, 'pago_mensal'] = dataset_full.loc[0, 'pago']
    if i != 0 and int(dataset_full.loc[i, 'mes']) - int(dataset_full.loc[i-1, 'mes'])
↳ != 1:
        dataset_full.loc[i, 'empenhado_mensal'] = dataset_full.loc[i, 'empenhado']
        dataset_full.loc[i, 'pago_mensal'] = dataset_full.loc[i, 'pago']

# --- Renomeando colunas --- #
dataset_full.rename(columns = {'empenhado': 'empenhado_acumulado',
↳ 'pago': 'pago_acumulado'}, inplace = True)

# --- Prints --- #
print(dataset_full.loc[:, ~dataset_full.columns.isin(['regiao',
↳ 'programa'])].head(10))

```

	periodo	ano	mes	tipo	cod_regiao	cod_programa	empenhado_acumulado	\
0	JUN_2012	2012	06	EQUIP	01	001	0.00	
1	JUL_2012	2012	07	EQUIP	01	001	0.00	
2	AGO_2012	2012	08	EQUIP	01	001	0.00	
3	SET_2012	2012	09	EQUIP	01	001	0.00	
4	OUT_2012	2012	10	EQUIP	01	001	0.00	
5	NOV_2012	2012	11	EQUIP	01	001	0.00	
6	DEZ_2012	2012	12	EQUIP	01	001	0.00	
7	JUN_2012	2012	06	EQUIP	01	003	16516129.06	
8	JUL_2012	2012	07	EQUIP	01	003	29763440.90	
9	AGO_2012	2012	08	EQUIP	01	003	29763440.90	

	pago_acumulado	empenhado_mensal	pago_mensal
0	0.00	0.00	0.00
1	0.00	0.00	0.00
2	0.00	0.00	0.00
3	0.00	0.00	0.00
4	0.00	0.00	0.00
5	0.00	0.00	0.00
6	0.00	0.00	0.00
7	16516129.06	16516129.06	16516129.06
8	25005350.58	13247311.84	8489221.52
9	29336897.53	0.00	4331546.95

Armazenamento dos Resultados

Após todo o tratamento aplicado aos dados, a base final, **dataset_full**, recebe um último tratamento antes dos dados serem finalmente armazenados em um arquivo excel (.xlsx).

A base de dados final passa por uma espécie de “verticalização”³ das informações. As colunas de valores agora são divididas em duas colunas. Enquanto os valores em si se alinham sob uma única coluna nomeada **valor**, uma coluna nomeada **categoria** é gerada para representar a categoria de investimento que está sendo tratada. Os campos período, ano, mês, tipo, código do programa, programa, código da região e região permanecem como variáveis categóricas.

Ao final da rotina é realizada uma limpeza no *environment* mantendo somente o dataframe final para visualização.

```
# ===== #
# === Armazenamento dos Resultados === #
# ===== #
if info_desired == 'p':

    # --- Ajuste vertical na estrutura dos dados --- #
    dataset_full = dataset_full.melt(
        id_vars = ['período', 'ano', 'mes', 'tipo', 'cod_programa', 'programa'],
        value_vars = ['empenhado_acumulado', 'pago_acumulado', 'empenhado_mensal',
↪ 'pago_mensal'],
        var_name = 'categoria',
        value_name = 'valor'
    )

    # --- Armazenando --- #
    with pd.ExcelWriter(path = 'investimentos_siof_ceara_programa.xlsx',
↪ engine='xlsxwriter') as writer:
```

³Esse procedimento é realizado por meio da função **melt**.

```

        dataset_full.to_excel(excel_writer = writer, sheet_name =
↪ 'investimentos_programa', index = False)

        # Rápida formatação na planilha
        workbook = writer.book
        worksheet = writer.sheets['investimentos_programa']
        money_formatting = workbook.add_format({'num_format': 'R$#,##0'})
        perc_formatting = workbook.add_format({'num_format': '0.0%'})
        worksheet.set_column('H:H', 15, money_formatting)
        #worksheet.set_column('K:L', 15, perc_formatting)
        #worksheet.set_column('A:F', 15)

        # --- Limpeza --- #
        del(dataset, folder_files, i, info_desired, writer, x, new, old, replacements,
↪ program_flag)#, money_formatting, perc_formatting, workbook, worksheet)

elif info_desired == 'r':

    # --- Ajuste vertical na estrutura dos dados --- #
    dataset_full = dataset_full.melt(
        id_vars = ['periodo', 'ano', 'mes', 'tipo', 'cod_regiao', 'regiao'],
        value_vars = ['empenhado_acumulado', 'pago_acumulado', 'empenhado_mensal',
↪ 'pago_mensal'],
        var_name = 'categoria',
        value_name = 'valor'
    )

    # --- Armazenando --- #
    with pd.ExcelWriter(path = 'investimentos_siof_ceara_regiao.xlsx',
↪ engine='xlsxwriter') as writer:
        dataset_full.to_excel(excel_writer = writer, sheet_name =
↪ 'investimentos_regiao', index = False)

        # Rápida formatação na planilha
        workbook = writer.book
        worksheet = writer.sheets['investimentos_regiao']
        money_formatting = workbook.add_format({'num_format': 'R$#,##0'})
        perc_formatting = workbook.add_format({'num_format': '0.0%'})
        worksheet.set_column('H:H', 15, money_formatting)
        #worksheet.set_column('K:L', 15, perc_formatting)
        #worksheet.set_column('A:F', 15)

        # --- Limpeza --- #
        del(dataset, folder_files, i, info_desired, writer, x, new, old, replacements,
↪ region_flag)#, money_formatting, perc_formatting, workbook, worksheet)

else:

```

```

# --- Ajuste vertical na estrutura dos dados --- #
dataset_full = dataset_full.melt(
    id_vars = ['periodo', 'ano', 'mes', 'tipo', 'cod_regiao', 'regiao',
↪ 'cod_programa', 'programa'],
    value_vars = ['empenhado_acumulado', 'pago_acumulado', 'empenhado_mensal',
↪ 'pago_mensal'],
    var_name = 'categoria',
    value_name = 'valor'
)

# --- Armazenando --- #
with pd.ExcelWriter(path = 'investimentos_siof_ceara_programa_regiao.xlsx',
↪ engine='xlsxwriter') as writer:
    dataset_full.to_excel(excel_writer = writer, sheet_name =
↪ 'investimentos_programa_regiao', index = False)

    # Rápida formatação na planilha
    workbook = writer.book
    worksheet = writer.sheets['investimentos_programa_regiao']
    money_formatting = workbook.add_format({'num_format': 'R$#,##0'})
    perc_formatting = workbook.add_format({'num_format': '0.0%'})
    worksheet.set_column('H:H', 15, money_formatting)
    #worksheet.set_column('K:L', 15, perc_formatting)
    #worksheet.set_column('A:F', 15)

# --- Limpeza --- #
del(dataset, folder_files, i, info_desired, writer, x, new, old, replacements,
↪ program_flag)

```